

Machine Learning for Physics I

Artificial neural networks, optimization, and automatic differentiation

Filippo Masi[★]

Laboratoire Jean Kuntzmann (LJK), TRIPOP team

Université Grenoble Alpes, Inria, MIAI | Artificial Intelligence & Mechanics (AIM)



[★]filippo.masi@inria.fr

Outline

- 1 Learning framework and artificial neural networks
- 2 Optimizing neural networks
- 3 Automatic differentiation
- 4 Training neural networks

Learning framework and artificial neural networks

Learning framework

A learning model (e.g. a neural network) is a finite-dimensional family of functions

$$\mathbf{f}_{\boldsymbol{\theta}} : \mathcal{X} \subset \mathbb{R}^{d_X} \rightarrow \mathcal{Y} \subset \mathbb{R}^{d_Y}, \quad \text{with } \boldsymbol{\theta} \in \Theta \subset \mathbb{R}^p \text{ model parameters.}$$

Given a labeled dataset $\mathcal{D}_{XY} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, with $\mathbf{x}_k \in \mathcal{X} \subset \mathbb{R}^{d_X}$ and $\mathbf{y}_k \in \mathcal{Y} \subset \mathbb{R}^{d_Y}$, the problem we aim to solve is the following problem

Learning framework

A learning model (e.g. a neural network) is a finite-dimensional family of functions

$$\mathbf{f}_\theta : \mathcal{X} \subset \mathbb{R}^{d_X} \rightarrow \mathcal{Y} \subset \mathbb{R}^{d_Y}, \quad \text{with } \theta \in \Theta \subset \mathbb{R}^p \text{ model parameters.}$$

Given a labeled dataset $\mathcal{D}_{XY} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, with $\mathbf{x}_k \in \mathcal{X} \subset \mathbb{R}^{d_X}$ and $\mathbf{y}_k \in \mathcal{Y} \subset \mathbb{R}^{d_Y}$, the problem we aim to solve is the following problem

Problem: empirical loss minimization

Find $\mathbf{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, $\mathbf{f}_\theta \in \mathcal{F}_\theta$ such that

$$\mathbf{f}_{\hat{\theta}}(\mathcal{D}_{XY}) = \arg \min_{\mathbf{f}_\theta \in \mathcal{F}_\theta} L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta), \quad L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{f}_\theta(\mathbf{x}_i), \mathbf{y}_i), \quad \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y}) \geq 0$$

where $\mathcal{F}_\theta$ is the class of regression functions, $L_{\mathcal{D}_{XY}}$ the empirical loss.

Learning framework

A learning model (e.g. a neural network) is a finite-dimensional family of functions

$$\mathbf{f}_\theta : \mathcal{X} \subset \mathbb{R}^{d_X} \rightarrow \mathcal{Y} \subset \mathbb{R}^{d_Y}, \quad \text{with } \theta \in \Theta \subset \mathbb{R}^p \text{ model parameters.}$$

Given a labeled dataset $\mathcal{D}_{XY} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, with $\mathbf{x}_k \in \mathcal{X} \subset \mathbb{R}^{d_X}$ and $\mathbf{y}_k \in \mathcal{Y} \subset \mathbb{R}^{d_Y}$, the problem we aim to solve is the following problem

Problem: empirical loss minimization

Find $\mathbf{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, $\mathbf{f}_\theta \in \mathcal{F}_\theta$ such that

$$\mathbf{f}_{\hat{\theta}}(\mathcal{D}_{XY}) = \arg \min_{\mathbf{f}_\theta \in \mathcal{F}_\theta} L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta), \quad L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{f}_\theta(\mathbf{x}_i), \mathbf{y}_i), \quad \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y}) \geq 0$$

where $\mathcal{F}_\theta$ is the class of regression functions, $L_{\mathcal{D}_{XY}}$ the empirical loss.

- ▶ $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$ is the "distance" between "true" labels and predictions, e.g.
- ▶ $\ell(\mathbf{f}_\theta(\mathbf{x}_i), \mathbf{y}_i) = \|\mathbf{f}_\theta(\mathbf{x}_i) - \mathbf{y}_i\|_2^2$.
- ▶ $L_{\mathcal{D}_{XY}}$ is an approximation of the "true" loss $L = \mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim p} [\ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y})]$, but $p(\mathbf{x}, \mathbf{y})$ is not known a priori.

Learning framework

A learning model (e.g. a neural network) is a finite-dimensional family of functions

$$\mathbf{f}_\theta : \mathcal{X} \subset \mathbb{R}^{d_X} \rightarrow \mathcal{Y} \subset \mathbb{R}^{d_Y}, \quad \text{with } \theta \in \Theta \subset \mathbb{R}^p \text{ model parameters.}$$

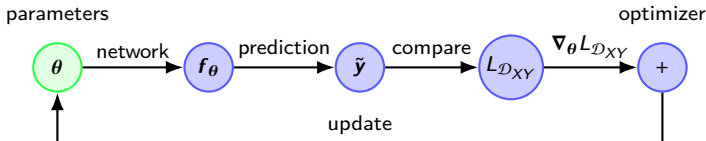
Given a labeled dataset $\mathcal{D}_{XY} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$, with $\mathbf{x}_k \in \mathcal{X} \subset \mathbb{R}^{d_X}$ and $\mathbf{y}_k \in \mathcal{Y} \subset \mathbb{R}^{d_Y}$, the problem we aim to solve is the following problem

Problem: empirical loss minimization

Find $\mathbf{f}_\theta : \mathcal{X} \rightarrow \mathcal{Y}$, $\mathbf{f}_\theta \in \mathcal{F}_\theta$ such that

$$\mathbf{f}_{\hat{\theta}}(\mathcal{D}_{XY}) = \arg \min_{\mathbf{f}_\theta \in \mathcal{F}_\theta} L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta), \quad L_{\mathcal{D}_{XY}}(\mathbf{f}_\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\mathbf{f}_\theta(\mathbf{x}_i), \mathbf{y}_i), \quad \ell(\mathbf{f}_\theta(\mathbf{x}), \mathbf{y}) \geq 0$$

where $\mathcal{F}_\theta$ is the class of regression functions, $L_{\mathcal{D}_{XY}}$ the empirical loss.



Example: least squares regression

Given observations $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, a linear regression model assumes that the relationship between the dependent variable y and the regressor vector $\mathbf{x}_i \in \mathbb{R}^{d_X}$ is linear, i.e.

$$\tilde{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}) = \theta_0 + \theta_1 \tilde{x}_i^{(1)} + \dots + \theta_{d_X} \tilde{x}_i^{(d_X)} = \tilde{\mathbf{x}}_i^\top \boldsymbol{\theta}, \quad \tilde{\mathbf{y}} = \mathbf{X} \boldsymbol{\theta}$$

with $\tilde{\mathbf{x}}_i = \left[1, x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(d_X)}\right]^\top$ and $\mathbf{X} = \left[\mathbf{x}_1^\top, \mathbf{x}_2^\top, \dots, \mathbf{x}_N^\top\right]^\top \in \mathbb{R}^{N \times (d_X+1)}$.

The optimum parameter vector is defined as the one that minimizes the sum of mean squared (empirical) loss:

$$\hat{\boldsymbol{\theta}} = \arg \min_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}), \quad L_{\mathcal{D}}(\boldsymbol{\theta}) = \frac{1}{2N} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2, \quad \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}} = \frac{1}{N} \mathbf{X}^\top (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

The optimum parameter vector is the solution of

$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta} - \mathbf{X}^\top \mathbf{y} = \mathbf{0}, \quad \text{or} \quad \hat{\boldsymbol{\theta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Note

In a neural network, the dependence of $f_{\boldsymbol{\theta}}$ on $\boldsymbol{\theta}$ is nonlinear, so the training problem is generally nonlinear and nonconvex.

The artificial neuron

Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s).

A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

The artificial neuron

Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s).

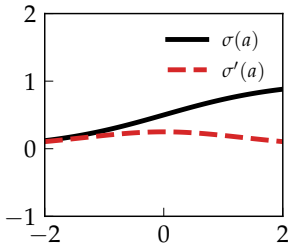
A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

◦ List of activation functions:

– sigmoid: $\sigma(a) = \frac{1}{1 + e^{-a}}$



The artificial neuron

Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s).

A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

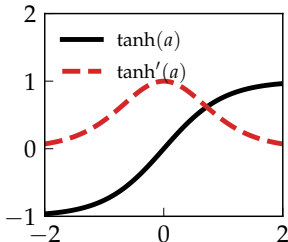
$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

◦ List of activation functions:

– sigmoid: $\sigma(a) = \frac{1}{1 + e^{-a}}$

– hyperbolic tangent: $\tanh(a) = \frac{e^{2a} - 1}{e^{2a} + 1}$



The artificial neuron

Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s).

A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

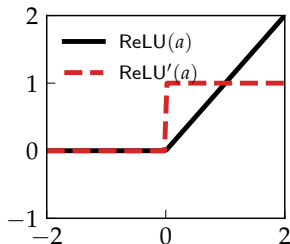
where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

◦ List of activation functions:

– sigmoid: $\sigma(a) = \frac{1}{1 + e^{-a}}$

– hyperbolic tangent: $\tanh(a) = \frac{e^{2a} - 1}{e^{2a} + 1}$

– rectified linear unit: $\text{ReLU}(a) = \max(0, a)$



The artificial neuron

Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s).

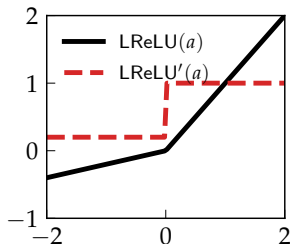
A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

◦ List of activation functions:

- sigmoid: $\sigma(a) = \frac{1}{1 + e^{-a}}$
- hyperbolic tangent: $\tanh(a) = \frac{e^{2a} - 1}{e^{2a} + 1}$
- rectified linear unit: $\text{ReLU}(a) = \max(0, a)$
- leaky ReLU: $\text{LReLU}(a) = \begin{cases} a, & a \geq 0 \\ \alpha a, & a < 0 \end{cases}$



The artificial neuron

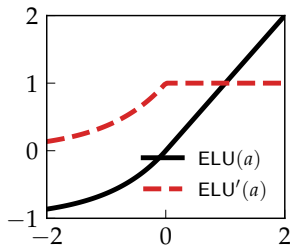
Artificial neurons have been formulated by the Pitts&McCulloch and Rosenblatt (1950s). A neuron performs the following nonlinear transformation on its inputs $\mathbf{x}$

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\mathbf{x})) = \mathcal{A}(\langle \mathbf{W}, \mathbf{x} \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot x_c + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where a is the linear pre-activation, $\mathbf{W}$ the weights, b the bias, and $\mathcal{A}$ the nonlinear activation function.

◦ List of activation functions:

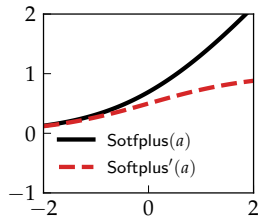
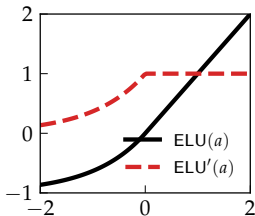
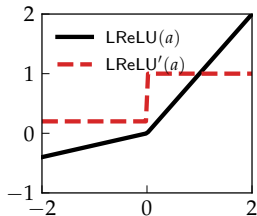
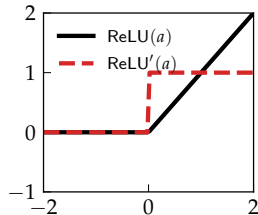
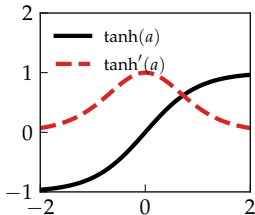
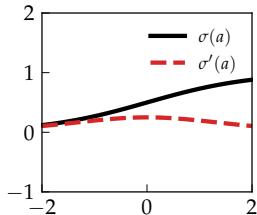
- sigmoid: $\sigma(a) = \frac{1}{1 + e^{-a}}$
- hyperbolic tangent: $\tanh(a) = \frac{e^{2a} - 1}{e^{2a} + 1}$
- rectified linear unit: $\text{ReLU}(a) = \max(0, a)$
- leaky ReLU: $\text{LReLU}(a) = \begin{cases} a, & a \geq 0 \\ \alpha a, & a < 0 \end{cases}$
- exponential linear unit: $\text{ELU}(a) = \begin{cases} a, & a \geq 0 \\ \alpha (e^a - 1), & a < 0 \end{cases}$



Activation functions

List of activation functions (non-exhaustive):

sigmoid, hyperbolic tangent, ReLU, LReLU, ELU, Softplus ($= \ln(1 + e^a)$).



The multilayer perceptron (MLP)

Stacking two layers of neurons yields an extremely flexible predictor model (MLP)

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\boldsymbol{\phi}(\mathbf{x}))) = \mathcal{A}(\langle \mathbf{W}, \boldsymbol{\phi}(\mathbf{x}) \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot \phi_c(\mathbf{x}) + b\right), \quad \theta := \{\mathbf{W}, b\}$$

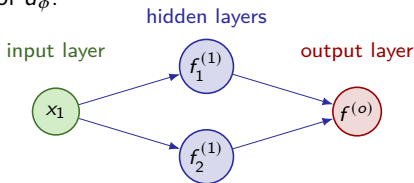
where $\boldsymbol{\phi} : \mathbf{x} \rightarrow \mathbb{R}^{d_{\phi}}$ is an intermediate transformation (hidden layer) mapping $\mathbf{x}$ into the space of hidden features of d_{ϕ} .

The multilayer perceptron (MLP)

Stacking two layers of neurons yields an extremely flexible predictor model (MLP)

$$f_{\theta}(\mathbf{x}) = \mathcal{A}(a(\boldsymbol{\phi}(\mathbf{x}))) = \mathcal{A}(\langle \mathbf{W}, \boldsymbol{\phi}(\mathbf{x}) \rangle + b) = \mathcal{A}\left(\sum_{c=1}^{d_X} W_c \cdot \phi_c(\mathbf{x}) + b\right), \quad \theta := \{\mathbf{W}, b\}$$

where $\boldsymbol{\phi} : \mathbf{x} \rightarrow \mathbb{R}^{d_{\phi}}$ is an intermediate transformation (hidden layer) mapping $\mathbf{x}$ into the space of hidden features of d_{ϕ} .



The neural network output reads (layers: one hidden $\mathbf{f}^{(1)}$ and output $\mathbf{f}^{(o)}$)

$$f_{\theta}(\mathbf{x}) = f^{(o)} \circ \mathbf{f}^{(1)}(\mathbf{x}) = \mathcal{A}^{(o)}(a^{(o)}(\mathbf{f}^{(1)}(\mathbf{x})))$$

$$= \mathcal{A}^{(o)}\left(\langle \mathbf{W}^{(o)}, \mathbf{f}^{(1)}(\mathbf{x}) \rangle + b^{(o)}\right) = \mathcal{A}^{(o)}\left(\sum_{c=1}^{d_{f_1}} W_c^{(o)} \cdot f_c^{(1)}(\mathbf{x}) + b^{(o)}\right)$$

$$\text{with } f_c^{(1)} = \mathcal{A}^{(1)}(a_c^{(1)}(\mathbf{x})) = \mathcal{A}^{(1)}\left(\langle \mathbf{W}^{(1)}, \mathbf{x} \rangle + b^{(1)}\right) = \mathcal{A}^{(1)}\left(\sum_{c=1}^{d_X} W_c^{(1)} \cdot x_c + b^{(1)}\right)$$

Stacking MLP: artificial neural networks

Using a MLP and stacking multiple hidden layers yields a (feed-forward) artificial neural network:

$$\begin{aligned}\mathbf{f}^{(0)} &= \mathbf{x}, \\ \mathbf{a}^{(\ell+1)} &= \langle \mathbf{W}^{(\ell)}, \mathbf{f}^{(\ell)} \rangle + \mathbf{b}^{(\ell)}, \\ \mathbf{f}^{(\ell+1)} &= \mathcal{A}_\ell \left(\mathbf{a}^{(\ell+1)} \right), && \text{hidden layer } \ell = 0, \dots, L-1, \\ \mathbf{f}_\theta(\mathbf{x}) &= \langle \mathbf{W}^{(o)}, \mathbf{f}^{(o)} \rangle + \mathbf{b}^{(o)}, && \text{output layer } (o),\end{aligned}$$

where the parameter vector is $\theta = \{ \mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}, \mathbf{W}^{(o)}, \mathbf{b}^{(o)} \}$.

Stacking MLP: artificial neural networks

Using a MLP and stacking multiple hidden layers yields a (feed-forward) artificial neural network:

$$\begin{aligned}\mathbf{f}^{(0)} &= \mathbf{x}, \\ \mathbf{a}^{(\ell+1)} &= \langle \mathbf{W}^{(\ell)}, \mathbf{f}^{(\ell)} \rangle + \mathbf{b}^{(\ell)}, \\ \mathbf{f}^{(\ell+1)} &= \mathcal{A}_\ell \left(\mathbf{a}^{(\ell+1)} \right), & \text{hidden layer } \ell = 0, \dots, L-1, \\ \mathbf{f}_\theta(\mathbf{x}) &= \langle \mathbf{W}^{(o)}, \mathbf{f}^{(o)} \rangle + \mathbf{b}^{(o)}, & \text{output layer } (o),\end{aligned}$$

where the parameter vector is $\theta = \{ \mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}, \mathbf{W}^{(o)}, \mathbf{b}^{(o)} \}$.

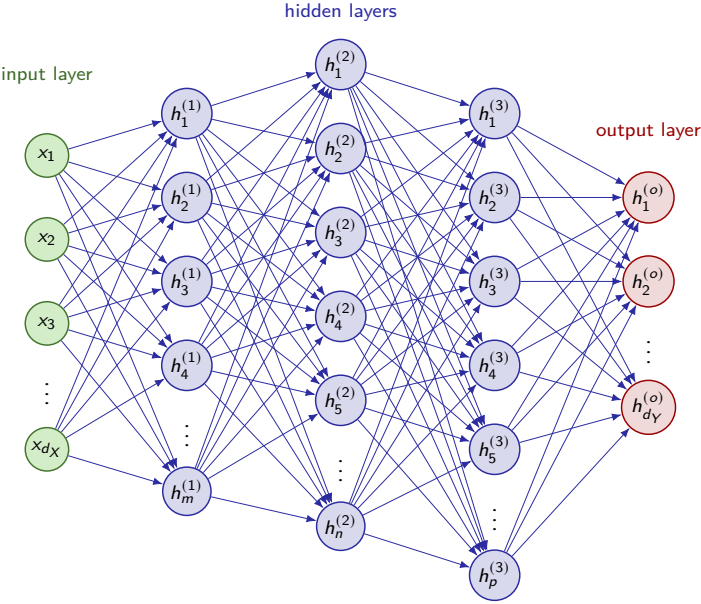
How many layers? Lower bounds for approximation by MLP neural nets

Given a function $h : [-T, T]^{d_X} \subset \mathcal{X} \rightarrow \mathbb{R}$, $T > 0$, $h \in W^{k,2}$, there exists a one-hidden-layer MLP $\hat{f}_\theta$ with sigmoid activation function and N_K hidden neurons such that the L^2 approximation error $e := \|h - \hat{f}_\theta\|_{L^2([-T, T]^{d_X})}$ is smaller than a prescribed tolerance ϵ if

$$N_K \approx \epsilon^{\frac{1-d_X}{k}} = \epsilon^{-\frac{d_X-1}{k}}, \quad d_X = \dim(\mathcal{X})$$

For $h : L^2[-T, T]^{d_X}$, to reduce the error by an order or magnitude, the number of neurons (width) must be multiplied by $10^{\frac{d_X-1}{2}}$

Stacking MLP: artificial neural networks



Optimizing neural networks

Gradient descent

Training seeks parameters $\hat{\theta} \in \arg \min_{\theta \in \Theta} L_{\mathcal{D}_{XY}}(\theta)$.

Throughout this section, the superscript (i) denotes the current optimization epoch/iteration. Starting from $\theta^{(0)}$, define

$$\mathbf{g}^{(i)} := \nabla_{\theta} L_{\mathcal{D}_{XY}}(\theta^{(i)}).$$

Gradient descent updates

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}, \quad \eta^{(i)} > 0.$$

Interpretation

The gradient points in the direction of steepest local increase of the loss. Its negative therefore points in the direction of steepest local decrease.

For a suitable learning rate,

$$L_{\mathcal{D}_{XY}}(\theta^{(i+1)}) < L_{\mathcal{D}_{XY}}(\theta^{(i)})$$

whenever $\mathbf{g}^{(i)} \neq \mathbf{0}$, at least locally.

Why gradient descent?

Let $L_{\mathcal{D}} : \Theta \rightarrow \mathbb{R}$ be differentiable and let $\Theta \subset \mathbb{R}^p$ be convex. If $\hat{\theta}$ is a local minimizer, then the first-order necessary condition is

$$\langle \nabla_{\theta} L_{\mathcal{D}}(\hat{\theta}), \theta - \hat{\theta} \rangle \geq 0, \quad \forall \theta \in \Theta.$$

If $L_{\mathcal{D}}$ is convex, this condition is also sufficient for a global minimizer.

At a current iterate $\theta^{(i)}$, a first-order Taylor expansion gives

$$L_{\mathcal{D}}(\theta^{(i)} + \delta\theta) = L_{\mathcal{D}}(\theta^{(i)}) + \langle \nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)}), \delta\theta \rangle + o(\|\delta\theta\|).$$

Among all perturbations of fixed norm $\|\delta\theta\| = r$,

$$\delta\theta^{\star} = -r \frac{\nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)})}{\|\nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)})\|}.$$

Steepest-descent update

Therefore, in the unconstrained case,

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)}).$$

If the gradient is nonzero, a sufficiently small $\eta^{(i)} > 0$ decreases the loss.

Stochastic Gradient Descent

The full empirical gradient at epoch i is

$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(\mathbf{f}_{\theta^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

Stochastic Gradient Descent

The full empirical gradient at epoch i is

$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\theta}} \ell(\mathbf{f}_{\boldsymbol{\theta}^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

◦ Gradient descent

$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}$$

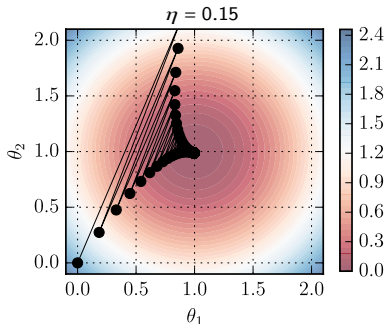
Stochastic Gradient Descent

The full empirical gradient at epoch i is

$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\theta}} \ell(\mathbf{f}_{\boldsymbol{\theta}^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

◦ Gradient descent

$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta \mathbf{g}^{(i)}$$



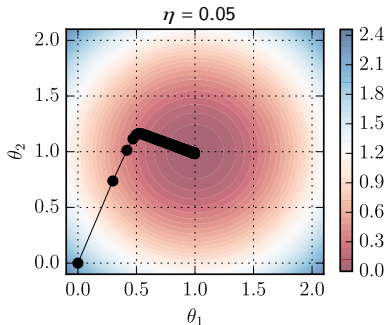
Stochastic Gradient Descent

The full empirical gradient at epoch i is

$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\boldsymbol{\theta}} \ell(\mathbf{f}_{\boldsymbol{\theta}^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

◦ Gradient descent

$$\boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta \mathbf{g}^{(i)}$$



Stochastic Gradient Descent

The full empirical gradient at epoch i is

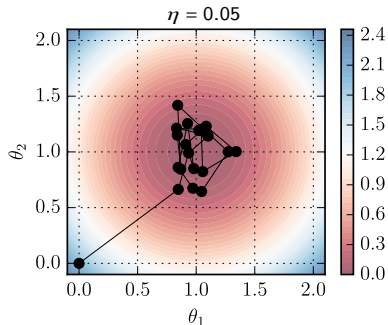
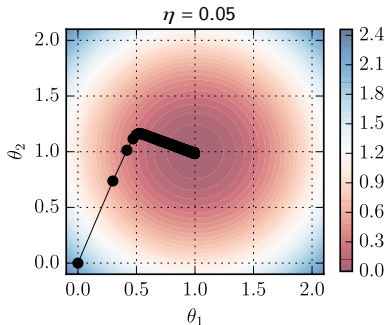
$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(\mathbf{f}_{\theta^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

◦ Gradient descent

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}$$

◦ Stochastic gradient descent

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \nabla_{\theta} \ell(\mathbf{f}_{\theta^{(i)}}(\mathbf{x}_n), \mathbf{y}_n)$$



Stochastic Gradient Descent

The full empirical gradient at epoch i is

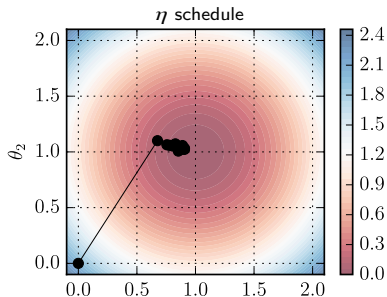
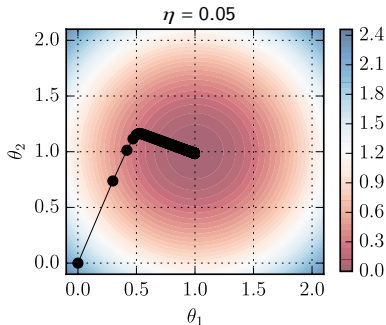
$$\mathbf{g}^{(i)} = \frac{1}{N} \sum_{n=1}^N \nabla_{\theta} \ell(\mathbf{f}_{\theta^{(i)}}(\mathbf{x}_n), \mathbf{y}_n) .$$

◦ Gradient descent

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \mathbf{g}^{(i)}$$

◦ Stochastic gradient descent

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \nabla_{\theta} \ell(\mathbf{f}_{\theta^{(i)}}(\mathbf{x}_n), \mathbf{y}_n)$$

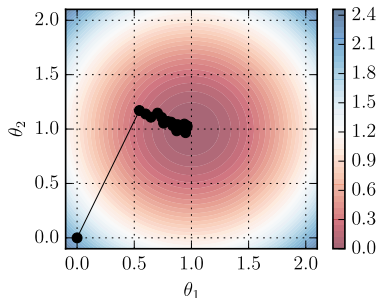


SGD algorithm

Given $\mathcal{D}_{XY} = \{(\mathbf{x}_n, \mathbf{y}_n)\}_{n=1}^N$, we optimize the parameters using mini-batches of size N_b .

Mini-batch stochastic gradient descent

- 1: Initialize parameters $\boldsymbol{\theta}^{(0)}$
- 2: **for** $i = 0, \dots, n_e - 1$ **do**
- 3: Randomly shuffle the indices $\{1, \dots, N\}$
- 4: Split them into mini-batches $\mathcal{B}_1, \dots, \mathcal{B}_M$, $M = \lceil N/N_b \rceil$
- 5: Set $\boldsymbol{\theta}^{(i,0)} \leftarrow \boldsymbol{\theta}^{(i)}$
- 6: **for** $m = 1, \dots, M$ **do**
- 7: Compute $\mathbf{g}^{(i,m)} \leftarrow \frac{1}{|\mathcal{B}_m|} \sum_{n \in \mathcal{B}_m} \nabla_{\boldsymbol{\theta}} \ell(\mathbf{f}_{\boldsymbol{\theta}^{(i,m-1)}}(\mathbf{x}_n), \mathbf{y}_n)$
- 8: Update $\boldsymbol{\theta}^{(i,m)} \leftarrow \boldsymbol{\theta}^{(i,m-1)} - \eta^{(i)} \mathbf{g}^{(i,m)}$
- 9: **end for**
- 10: Set $\boldsymbol{\theta}^{(i+1)} \leftarrow \boldsymbol{\theta}^{(i,M)}$
- 11: **end for**



Learning-rate scheduling

The learning rate $\eta^{(i)}$ controls the size of each parameter update. A fixed value is rarely optimal throughout training.

Too large

- ▶ unstable iterates;
- ▶ oscillation around sharp valleys;
- ▶ possible numerical overflow or NaNs.

Too small

- ▶ slow convergence;
- ▶ early apparent stagnation;
- ▶ poor practical use of compute time.

Common schedules start larger and decay later, for instance

$$\eta^{(i)} = \eta_0 0.1^{i/s} \quad \text{or} \quad \eta^{(i)} = \begin{cases} \eta_0, & i < i_1, \\ \eta_1, & i_1 \leq i < i_2, \\ \eta_2, & i \geq i_2. \end{cases}$$

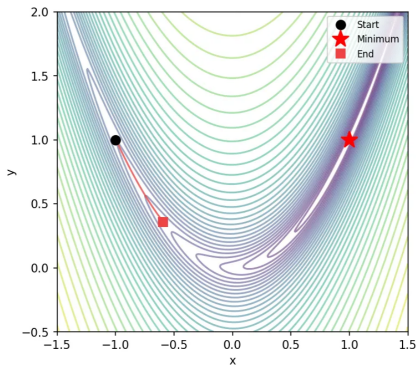
Gradient descent with momentum

Let $\mathbf{g}^{(i)} = \nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)})$. Momentum uses an exponentially weighted memory of previous gradients:

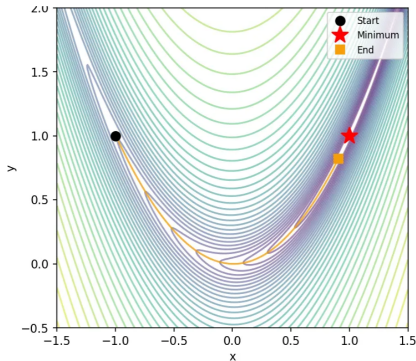
$$\mathbf{v}^{(i+1)} = \gamma \mathbf{v}^{(i)} + \mathbf{g}^{(i)}, \quad \theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \mathbf{v}^{(i+1)}, \quad 0 \leq \gamma < 1.$$

Equivalently, one may absorb the learning rate into the velocity update.

SGD (2000 steps, dist = 1.72)



SGD with momentum (2000 steps, dist = 0.2)



<https://sesen.ai/blog/optimisers-sgd-momentum-adam-from-scratch>

Adaptive optimizers: Adagrad and RMSProp

Let $\mathbf{g}^{(i)} = \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$. The vector $\mathbf{s}$ stores component-wise squared-gradient history.¹

Adaptive Gradient (Adagrad)

Adapt the learning rate based on the history of its squared gradients.

Initialize $\mathbf{s}^{(0)} = \mathbf{0}$. For $i \geq 1$,

$$\mathbf{s}^{(i)} = \mathbf{s}^{(i-1)} + \mathbf{g}^{(i)} \odot \mathbf{g}^{(i)}, \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta^{(i)} \frac{\mathbf{g}^{(i)}}{\sqrt{\mathbf{s}^{(i)} + \epsilon}}.$$

Root Mean Square Propagation (RMSProp)

Popular variation of Adagrad, which introduces exponential decay, so that older gradients are “forgotten”.

With $0 < \beta < 1$,

$$\mathbf{s}^{(i)} = \beta \mathbf{s}^{(i-1)} + (1 - \beta) \mathbf{g}^{(i)} \odot \mathbf{g}^{(i)}, \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} - \eta^{(i)} \frac{\mathbf{g}^{(i)}}{\sqrt{\mathbf{s}^{(i)} + \epsilon}}.$$

If $\beta = 0$, RMSprop reduces to standard GD method, while $\beta \geq 1$ leads to Adagrad.

¹All products, divisions, and square roots below are component-wise. Here, $\epsilon > 0$ prevents division by zero.

Adaptive momentum optimizer: Adam

Adam is a hybrid between momentum and Adagrad and combines a momentum-like first moment with an adaptive second moment. Let $\mathbf{g}^{(i)} = \nabla_{\theta} L_{\mathcal{D}}(\theta^{(i)})$ and initialize $\mathbf{m}^{(0)} = \mathbf{s}^{(0)} = \mathbf{0}$.

$$\mathbf{m}^{(i)} = \beta_1 \mathbf{m}^{(i-1)} + (1 - \beta_1) \mathbf{g}^{(i)},$$

$$\mathbf{s}^{(i)} = \beta_2 \mathbf{s}^{(i-1)} + (1 - \beta_2) \mathbf{g}^{(i)} \odot \mathbf{g}^{(i)},$$

$$\widehat{\mathbf{m}}^{(i)} = \frac{\mathbf{m}^{(i)}}{1 - \beta_1^i}, \quad \widehat{\mathbf{s}}^{(i)} = \frac{\mathbf{s}^{(i)}}{1 - \beta_2^i},$$

$$\theta^{(i+1)} = \theta^{(i)} - \eta^{(i)} \frac{\widehat{\mathbf{m}}^{(i)}}{\sqrt{\widehat{\mathbf{s}}^{(i)} + \epsilon}}.$$

Common choices

$0 < \beta_1, \beta_2 < 1$, typically $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

From gradient descent to Newton's method

At epoch i , let $\mathbf{g}^{(i)} := \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$. Gradient descent uses only the local slope:

$$\mathbf{d}_{\text{GD}}^{(i)} = -\mathbf{g}^{(i)} \quad \implies \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} + \eta^{(i)} \mathbf{d}_{\text{GD}}^{(i)}.$$

From gradient descent to Newton's method

At epoch i , let $\mathbf{g}^{(i)} := \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$. Gradient descent uses only the local slope:

$$\mathbf{d}_{\text{GD}}^{(i)} = -\mathbf{g}^{(i)} \quad \implies \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} + \eta^{(i)} \mathbf{d}_{\text{GD}}^{(i)}.$$

A second-order approximation of the loss around $\boldsymbol{\theta}^{(i)}$ is

$$L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)} + \mathbf{d}) \approx L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)}) + (\mathbf{g}^{(i)})^{\top} \mathbf{d} + \frac{1}{2} \mathbf{d}^{\top} \mathbf{H}^{(i)} \mathbf{d},$$

where $\mathbf{H}^{(i)} := \nabla_{\boldsymbol{\theta}}^2 L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$ is the Hessian.

From gradient descent to Newton's method

At epoch i , let $\mathbf{g}^{(i)} := \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$. Gradient descent uses only the local slope:

$$\mathbf{d}_{\text{GD}}^{(i)} = -\mathbf{g}^{(i)} \quad \implies \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} + \eta^{(i)} \mathbf{d}_{\text{GD}}^{(i)}.$$

A second-order approximation of the loss around $\boldsymbol{\theta}^{(i)}$ is

$$L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)} + \mathbf{d}) \approx L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)}) + (\mathbf{g}^{(i)})^{\top} \mathbf{d} + \frac{1}{2} \mathbf{d}^{\top} \mathbf{H}^{(i)} \mathbf{d},$$

where $\mathbf{H}^{(i)} := \nabla_{\boldsymbol{\theta}}^2 L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$ is the Hessian.

Minimizing this quadratic model means solving $\mathbf{H}^{(i)} \mathbf{d}_{\text{N}}^{(i)} = -\mathbf{g}^{(i)}$.

From gradient descent to Newton's method

At epoch i , let $\mathbf{g}^{(i)} := \nabla_{\boldsymbol{\theta}} L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$. Gradient descent uses only the local slope:

$$\mathbf{d}_{\text{GD}}^{(i)} = -\mathbf{g}^{(i)} \quad \implies \quad \boldsymbol{\theta}^{(i+1)} = \boldsymbol{\theta}^{(i)} + \eta^{(i)} \mathbf{d}_{\text{GD}}^{(i)}.$$

A second-order approximation of the loss around $\boldsymbol{\theta}^{(i)}$ is

$$L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)} + \mathbf{d}) \approx L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)}) + (\mathbf{g}^{(i)})^{\top} \mathbf{d} + \frac{1}{2} \mathbf{d}^{\top} \mathbf{H}^{(i)} \mathbf{d},$$

where $\mathbf{H}^{(i)} := \nabla_{\boldsymbol{\theta}}^2 L_{\mathcal{D}}(\boldsymbol{\theta}^{(i)})$ is the Hessian.

Minimizing this quadratic model means solving $\mathbf{H}^{(i)} \mathbf{d}_{\text{N}}^{(i)} = -\mathbf{g}^{(i)}$.

Why not use Newton's method directly?

For a network with p parameters, $\mathbf{H}^{(i)} \in \mathbb{R}^{p \times p}$. Explicitly forming and storing a dense Hessian requires $\mathcal{O}(p^2)$ memory. Solving the Newton system can also be costly. Moreover, for a nonconvex loss, $\mathbf{H}^{(i)}$ need not be positive definite, so the Newton direction need not be a descent direction.

Broyden-Fletcher-Goldfarb-Shanno algorithm

Suppose one optimization step has produced $\boldsymbol{\theta}^{(i+1)}$ and define

$$\mathbf{s}^{(i)} := \boldsymbol{\theta}^{(i+1)} - \boldsymbol{\theta}^{(i)}, \quad \mathbf{y}^{(i)} := \mathbf{g}^{(i+1)} - \mathbf{g}^{(i)}.$$

Broyden-Fletcher-Goldfarb-Shanno algorithm

Suppose one optimization step has produced $\boldsymbol{\theta}^{(i+1)}$ and define

$$\mathbf{s}^{(i)} := \boldsymbol{\theta}^{(i+1)} - \boldsymbol{\theta}^{(i)}, \quad \mathbf{y}^{(i)} := \mathbf{g}^{(i+1)} - \mathbf{g}^{(i)}.$$

A first-order expansion of the gradient gives $\mathbf{g}^{(i+1)} \approx \mathbf{g}^{(i)} + \mathbf{H}^{(i)} \mathbf{s}^{(i)}$, and therefore $\mathbf{y}^{(i)} \approx \mathbf{H}^{(i)} \mathbf{s}^{(i)}$. Thus each pair $(\mathbf{s}^{(i)}, \mathbf{y}^{(i)})$ contains information about the local curvature of the loss.

Broyden-Fletcher-Goldfarb-Shanno algorithm

Suppose one optimization step has produced $\theta^{(i+1)}$ and define

$$\mathbf{s}^{(i)} := \theta^{(i+1)} - \theta^{(i)}, \quad \mathbf{y}^{(i)} := \mathbf{g}^{(i+1)} - \mathbf{g}^{(i)}.$$

A first-order expansion of the gradient gives $\mathbf{g}^{(i+1)} \approx \mathbf{g}^{(i)} + \mathbf{H}^{(i)} \mathbf{s}^{(i)}$, and therefore $\mathbf{y}^{(i)} \approx \mathbf{H}^{(i)} \mathbf{s}^{(i)}$. Thus each pair $(\mathbf{s}^{(i)}, \mathbf{y}^{(i)})$ contains information about the local curvature of the loss.

Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm

Construct an approximation $\mathbf{B}^{(i)} \approx (\mathbf{H}^{(i)})^{-1}$ such that $\mathbf{B}^{(i+1)} \mathbf{y}^{(i)} = \mathbf{s}^{(i)}$. With $\rho^{(i)} := 1/((\mathbf{y}^{(i)})^\top \mathbf{s}^{(i)})$,

$$\begin{aligned} \mathbf{B}^{(i+1)} &= \left(\mathbf{I} - \rho^{(i)} \mathbf{s}^{(i)} (\mathbf{y}^{(i)})^\top \right) \mathbf{B}^{(i)} \left(\mathbf{I} - \rho^{(i)} \mathbf{y}^{(i)} (\mathbf{s}^{(i)})^\top \right) + \rho^{(i)} \mathbf{s}^{(i)} (\mathbf{s}^{(i)})^\top, \\ \mathbf{d}^{(i+1)} &= -\mathbf{B}^{(i+1)} \mathbf{g}^{(i+1)}. \end{aligned}$$

Broyden-Fletcher-Goldfarb-Shanno algorithm

Suppose one optimization step has produced $\theta^{(i+1)}$ and define

$$\mathbf{s}^{(i)} := \theta^{(i+1)} - \theta^{(i)}, \quad \mathbf{y}^{(i)} := \mathbf{g}^{(i+1)} - \mathbf{g}^{(i)}.$$

A first-order expansion of the gradient gives $\mathbf{g}^{(i+1)} \approx \mathbf{g}^{(i)} + \mathbf{H}^{(i)} \mathbf{s}^{(i)}$, and therefore $\mathbf{y}^{(i)} \approx \mathbf{H}^{(i)} \mathbf{s}^{(i)}$. Thus each pair $(\mathbf{s}^{(i)}, \mathbf{y}^{(i)})$ contains information about the local curvature of the loss.

Broyden-Fletcher-Goldfarb-Shanno (BFGS) algorithm

Construct an approximation $\mathbf{B}^{(i)} \approx (\mathbf{H}^{(i)})^{-1}$ such that $\mathbf{B}^{(i+1)} \mathbf{y}^{(i)} = \mathbf{s}^{(i)}$. With $\rho^{(i)} := 1/((\mathbf{y}^{(i)})^\top \mathbf{s}^{(i)})$,

$$\begin{aligned} \mathbf{B}^{(i+1)} &= \left(\mathbf{I} - \rho^{(i)} \mathbf{s}^{(i)} (\mathbf{y}^{(i)})^\top \right) \mathbf{B}^{(i)} \left(\mathbf{I} - \rho^{(i)} \mathbf{y}^{(i)} (\mathbf{s}^{(i)})^\top \right) + \rho^{(i)} \mathbf{s}^{(i)} (\mathbf{s}^{(i)})^\top, \\ \mathbf{d}^{(i+1)} &= -\mathbf{B}^{(i+1)} \mathbf{g}^{(i+1)}. \end{aligned}$$

Limited memory algorithm (L-BFGS)

Full BFGS stores $\mathbf{B}^{(i)} \in \mathbb{R}^{p \times p}$. L-BFGS stores only the most recent m curvature pairs $\{(\mathbf{s}^{(j)}, \mathbf{y}^{(j)})\}_{j=i-m}^{i-1}$ and uses them to compute products $\mathbf{B}^{(i)} \mathbf{g}^{(i)}$.

$$\text{memory: } O(mp) \quad \text{instead of} \quad O(p^2), \quad m \ll p.$$

Automatic differentiation

Differentiation in scientific computing

- ▶ Automatic Differentiation (AD) is the efficient calculation of derivatives of numerical algorithms (including computer programs), via the use of the chain rule of calculus implemented on a computer.
- ▶ Differentiation (derivatives, gradients, Jacobians, and Hessians) is used throughout scientific computation. It is required in numerical methods for, among other applications: nonlinear optimization (including training neural networks)
 - solution of nonlinear system of equations
 - solution of implicit equations
 - solution of ODEs/PDEs.
- ▶ There are three automatic methods to perform differentiation in scientific computation:
 - finite-difference approximation: inefficient approximation
 - symbolic differentiation: *expression swell*²
 - automatic differentiation (AD).

²Symbolic differentiation can easily produce exponentially large symbolic expressions.

Finite-difference algorithm

Given a function (or numerical algorithm) $f : \mathbb{R}^n \rightarrow \mathbb{R}$, its partial derivatives at $\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$ can be approximated as

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h},$$

where $h > 0$ is a small step size and $\mathbf{e}_i \in \mathbb{R}^n$ is unit vector with 1 in the i -th position, and zero otherwise.

Assuming smoothness of f , Taylor series expansion gives

$$f(\mathbf{x} + h\mathbf{e}_i) = f(\mathbf{x}) + h \frac{\partial f(\mathbf{x})}{\partial x_i} + O(h^2).$$

Dividing by h and rearranging gives

$$\frac{\partial f(\mathbf{x})}{\partial x_i} - \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} = O(h),$$

i.e., the approximation has truncation error $O(h)$.

- ▶ The error disappears as $h \rightarrow 0$ (in theory), but does not account for the round-off error (adding a large number x_i to a small number h), e.g. in a hypothetical 4-digit machine $10.22 + 0.001258 = 10.22$.

Approximation $\neq$ machine precision.

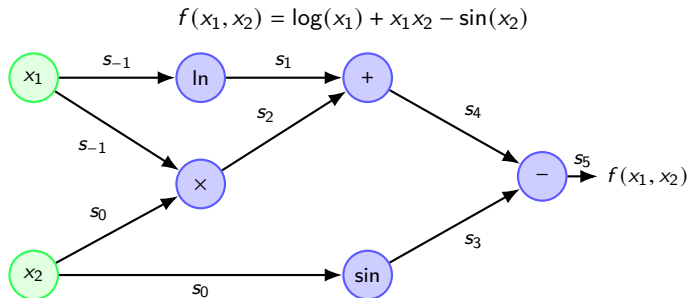
- ▶ It is not efficient in high dimensions (the number of function evaluations grows with n).

Automatic differentiation

- ▶ AD is a method that produces numerical derivatives for arbitrarily complex functions (or programs) at machine precision (cf. finite-difference approximations.)
- ▶ The necessary assumption is that the function or computation can be organized into a sequence of simpler primitives for which derivatives are available directly.
AD accumulates information in forward mode or reverse mode using the chain rule of calculus.

Automatic differentiation

- ▶ AD is a method that produces numerical derivatives for arbitrarily complex functions (or programs) at machine precision (cf. finite-difference approximations.)
- ▶ The necessary assumption is that the function or computation can be organized into a sequence of simpler primitives for which derivatives are available directly.
AD accumulates information in forward mode or reverse mode using the chain rule of calculus.



Automatic differentiation

- ▶ AD is a method that produces numerical derivatives for arbitrarily complex functions (or programs) at machine precision (cf. finite-difference approximations.)
- ▶ The necessary assumption is that the function or computation can be organized into a sequence of simpler primitives for which derivatives are available directly.
AD accumulates information in forward mode or reverse mode using the chain rule of calculus.

$$f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2)$$

$$s_{-1} = x_1,$$

$$s_0 = x_2,$$

$$s_1 = \log(s_{-1}),$$

$$s_2 = s_{-1} \cdot s_0,$$

$$s_3 = \sin(s_0),$$

$$s_4 = s_1 + s_2,$$

$$s_5 = s_4 - s_3 = f(x_1, x_2).$$

Forward-mode automatic differentiation

Suppose we want to compute the gradient of $f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2)$ at $(x_1, x_2) = (2, 5)$:

$$\nabla_{\mathbf{x}} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right).$$

Computing the two components separately requires two forward passes.

Forward-mode automatic differentiation

Suppose we want to compute the gradient of $f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2)$ at $(x_1, x_2) = (2, 5)$:

$$\nabla_{\mathbf{x}} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right).$$

Computing the two components separately requires two forward passes.

To compute $\partial f / \partial x_1$, seed the input tangents as $\dot{x}_1 = 1$, $\dot{x}_2 = 0$, producing the following computation (on a 4-digit machine):

forward primal trace				primal tangent trace			
$s_{-1} = x_1$	=	2		$\dot{s}_{-1} = \dot{x}_1$	=	1	
$s_0 = x_2$	=	5		$\dot{s}_0 = \dot{x}_2$	=	0	
$s_1 = \log s_{-1}$	=	0.6931		$\dot{s}_1 = \dot{s}_{-1} / s_{-1}$	=	0.5	
$s_2 = s_{-1} s_0$	=	10		$\dot{s}_2 = s_0 \dot{s}_{-1} + s_{-1} \dot{s}_0$	=	5	
$s_3 = \sin s_0$	=	-0.9589		$\dot{s}_3 = \cos(s_0) \dot{s}_0$	=	0	
$s_4 = s_1 + s_2$	=	10.69		$\dot{s}_4 = \dot{s}_1 + \dot{s}_2$	=	5.5	
$s_5 = s_4 - s_3$	=	11.65		$\dot{s}_5 = \dot{s}_4 - \dot{s}_3$	=	5.5	
$f = s_5$	=	11.65		$\partial f / \partial x_1 = \dot{s}_5$	=	5.5	

Forward-mode automatic differentiation

Suppose we want to compute the gradient of $f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2)$ at $(x_1, x_2) = (2, 5)$:

$$\nabla_{\mathbf{x}} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right).$$

Computing the two components separately requires two forward passes.

To compute $\partial f / \partial x_1$, seed the input tangents as $\dot{x}_1 = 1$, $\dot{x}_2 = 0$, producing the following computation (on a 4-digit machine):

forward primal trace			primal tangent trace		
$s_{-1} = x_1$	=	2	$\dot{s}_{-1} = \dot{x}_1$	=	1
$s_0 = x_2$	=	5	$\dot{s}_0 = \dot{x}_2$	=	0
$s_1 = \log s_{-1}$	=	0.6931	$\dot{s}_1 = \dot{s}_{-1} / s_{-1}$	=	0.5
$s_2 = s_{-1} s_0$	=	10	$\dot{s}_2 = s_0 \dot{s}_{-1} + s_{-1} \dot{s}_0$	=	5
$s_3 = \sin s_0$	=	-0.9589	$\dot{s}_3 = \cos(s_0) \dot{s}_0$	=	0
$s_4 = s_1 + s_2$	=	10.69	$\dot{s}_4 = \dot{s}_1 + \dot{s}_2$	=	5.5
$s_5 = s_4 - s_3$	=	11.65	$\dot{s}_5 = \dot{s}_4 - \dot{s}_3$	=	5.5
$f = s_5$	=	11.65	$\partial f / \partial x_1 = \dot{s}_5$	=	5.5

Analytically, $\partial f / \partial x_1 = 1/x_1 + x_2$, and $\partial f / \partial x_1(2, 5) = \frac{1}{2} + 5 = 5.5$ (AD is exact, in this case)

Forward-mode automatic differentiation

Suppose we want to compute the gradient of $f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2)$ at $(x_1, x_2) = (2, 5)$:

$$\nabla_{\mathbf{x}} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right).$$

Computing the two components separately requires two forward passes.

To compute $\partial f / \partial x_2$, seed the input tangents as $\dot{x}_1 = 0$, $\dot{x}_2 = 1$, producing the following computation (on a 4-digit machine):

forward primal trace				primal tangent trace			
$s_{-1} = x_1$	=	2		$\dot{s}_{-1} = \dot{x}_1$	=	0	
$s_0 = x_2$	=	5		$\dot{s}_0 = \dot{x}_2$	=	1	
$s_1 = \log s_{-1}$	=	0.6931		$\dot{s}_1 = \dot{s}_{-1} / s_{-1}$	=	0	
$s_2 = s_{-1} s_0$	=	10		$\dot{s}_2 = s_0 \dot{s}_{-1} + s_{-1} \dot{s}_0$	=	2	
$s_3 = \sin s_0$	=	-0.9589		$\dot{s}_3 = \cos(s_0) \dot{s}_0$	=	0.2837	
$s_4 = s_1 + s_2$	=	10.69		$\dot{s}_4 = \dot{s}_1 + \dot{s}_2$	=	2	
$s_5 = s_4 - s_3$	=	11.65		$\dot{s}_5 = \dot{s}_4 - \dot{s}_3$	=	1.716	
$f = s_5$	=	11.65		$\partial f / \partial x_1 = \dot{s}_5$	=	1.716	

Analytically, $\partial f / \partial x_2 = x_1 - \cos(x_2)$, $\partial f / \partial x_2(2, 5) = 2 - \cos(2) = \underline{1.7163378} \dots$ (machine precision)

Forward-mode automatic differentiation

Given a function $y = f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^n$, the computation of the gradient

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

requires to run n separate passes of forward AD. So it looks like this has the same problem of finite-difference approximations: it scales poorly with the dimension of the input.

However, for a vector function $\mathbf{y} = \mathbf{f}(\mathbf{x})$, with $\mathbf{y} \in \mathbb{R}^m$, one is able to compute the derivatives $\nabla_{x_i} \mathbf{f}$ in a single pass.

Forward-mode automatic differentiation

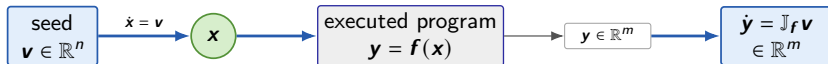
Given a function $y = f(\mathbf{x})$ with $\mathbf{x} \in \mathbb{R}^n$, the computation of the gradient

$$\nabla_{\mathbf{x}} f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)$$

requires to run n separate passes of forward AD. So it looks like this has the same problem of finite-difference approximations: it scales poorly with the dimension of the input.

However, for a vector function $\mathbf{y} = \mathbf{f}(\mathbf{x})$, with $\mathbf{y} \in \mathbb{R}^m$, one is able to compute the derivatives $\nabla_{x_i} \mathbf{f}$ in a single pass.

Jacobian-vector product (JVP): input-seeded directional derivative



$$\text{JVP}_f(\mathbf{x}; \mathbf{v}) = \mathbb{J}_f(\mathbf{x}) \mathbf{v} = \left. \frac{d}{d\epsilon} \mathbf{f}(\mathbf{x} + \epsilon \mathbf{v}) \right|_{\epsilon=0}$$

Columns of the Jacobian: seeds $\mathbf{v} = \mathbf{e}_j$ return column j (full Jacobian costs $\approx n$ sweeps).

- the Jacobian $\mathbb{J}_f$ of $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ requires n passes. If $m \gg n$ we are okay, but what if $n \gg m$?

Reverse-mode automatic differentiation

When there are many more inputs than outputs ($n > m$), the solution is to run the process backwards, from the output to the input.

Reverse-mode automatic differentiation uses the notion of adjoint

$$\bar{s}_i = \frac{\partial y}{\partial s_i}$$

to each intermediate variable s_i , where y is the function output to be differentiated. These adjoints can be computed in reverse order using the chain rule, until the adjoints $\bar{s}_{-n+1}, \dots, \bar{s}_0$ for the inputs are reached (these are the sought derivatives).

Reverse-mode automatic differentiation

When there are many more inputs than outputs ($n > m$), the solution is to run the process backwards, from the output to the input.

Reverse-mode automatic differentiation uses the notion of adjoint

$$\bar{s}_i = \frac{\partial y}{\partial s_i}$$

to each intermediate variable s_i , where y is the function output to be differentiated. These adjoints can be computed in reverse order using the chain rule, until the adjoints $\bar{s}_{-n+1}, \dots, \bar{s}_0$ for the inputs are reached (these are the sought derivatives).

Therefore, the gradient of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be computed in a single backward pass, but a forward pass is still necessary to populate the values of the intermediate variables for use in the reverse pass.

Reverse-mode automatic differentiation

When there are many more inputs than outputs ($n > m$), the solution is to run the process backwards, from the output to the input.

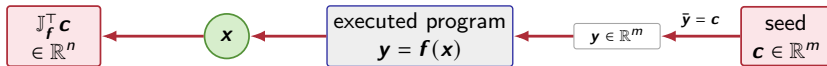
Reverse-mode automatic differentiation uses the notion of adjoint

$$\bar{s}_i = \frac{\partial y}{\partial s_i}$$

to each intermediate variable s_i , where y is the function output to be differentiated. These adjoints can be computed in reverse order using the chain rule, until the adjoints $\bar{s}_{-n+1}, \dots, \bar{s}_0$ for the inputs are reached (these are the sought derivatives).

Therefore, the gradient of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ can be computed in a single backward pass, but a forward pass is still necessary to populate the values of the intermediate variables for use in the reverse pass.

vector-Jacobian product (VJP): output-seeded sensitivity



$$\text{VJP}_f(\mathbf{x}; \mathbf{c}) = \mathbf{c}^\top \mathbb{J}_f(\mathbf{x}) \iff \mathbb{J}_f(\mathbf{x})^\top \mathbf{c} \text{ as a column}$$

Rows of the Jacobian: seeds $\mathbf{c} = \mathbf{e}_i$ select output i (full Jacobian costs $\approx m$ sweeps).

Reverse-mode automatic differentiation

Consider again

$$y = f(x_1, x_2) = \log(x_1) + x_1 x_2 - \sin(x_2), \quad (x_1, x_2) = (2, 5).$$

The full gradient can be computed with a single backward pass. Define the adjoint

$$\bar{s}_j := \frac{\partial y}{\partial s_j}.$$

forward primal trace			reverse adjoint trace		
$y = s_5$	=	11.65			
$s_5 = s_4 - s_3$	=	11.65	$\bar{s}_5 = \partial y / \partial s_5$	=	1
$s_4 = s_1 + s_2$	=	10.69	$\bar{s}_4 = \bar{s}_5 \partial s_5 / \partial s_4 = \bar{s}_5 \cdot 1$	=	1
$s_3 = \sin(s_0)$	=	-0.9589	$\bar{s}_3 = \bar{s}_5 \partial s_5 / \partial s_3 = \bar{s}_5 \cdot (-1)$	=	-1
$s_2 = s_{-1} s_0$	=	10	$\bar{s}_2 = \bar{s}_4 \partial s_4 / \partial s_2 = \bar{s}_4 \cdot 1$	=	1
$s_1 = \log(s_{-1})$	=	0.6931	$\bar{s}_1 = \bar{s}_4 \partial s_4 / \partial s_1 = \bar{s}_4 \cdot 1$	=	1
$s_0 = x_2$	=	5	$\bar{s}_0 = \bar{s}_2 \partial s_2 / \partial s_0 + \bar{s}_3 \partial s_3 / \partial s_0 = \bar{s}_2 s_{-1} + \bar{s}_3 \cos(s_0)$	=	1.716
$s_{-1} = x_1$	=	2	$\bar{s}_{-1} = \bar{s}_2 \partial s_2 / \partial s_{-1} + \bar{s}_1 \partial s_1 / \partial s_{-1} = \bar{s}_2 s_0 + \bar{s}_1 / s_{-1}$	=	5.5
			$\bar{x}_2 = \bar{s}_0$	=	1.7163
			$\bar{x}_1 = \bar{s}_{-1}$	=	5.5

Since $s_{-1} = x_1$ and $s_0 = x_2$,

$$\nabla f(2, 5) = (\bar{s}_{-1}, \bar{s}_0) = (5.5, 1.716).$$

JVP and VJP are different Jacobian contractions

$$\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \nabla_{\mathbf{x}} \mathbf{f} = \mathbf{J}_{\mathbf{f}}(\mathbf{x}) \in \mathbb{R}^{m \times n}$$

JVP and VJP are different Jacobian contractions

$$\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \nabla_{\mathbf{x}} \mathbf{f} = \mathbf{J}_{\mathbf{f}}(\mathbf{x}) \in \mathbb{R}^{m \times n}$$

$$\mathbf{J}_{\mathbf{f}} \mathbf{v}$$

JVP — forward mode

seed $\mathbf{v} \in \mathbb{R}^n$ at the input, result in $\mathbb{R}^m$
directional derivative / Jacobian columns
full Jacobian: about n sweeps

JVP and VJP are different Jacobian contractions

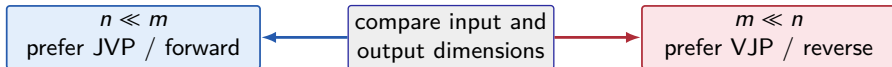
$$\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m, \quad \nabla_{\mathbf{x}} \mathbf{f} = \mathbf{J}_{\mathbf{f}}(\mathbf{x}) \in \mathbb{R}^{m \times n}$$

$\mathbf{J}_{\mathbf{f}} \mathbf{v}$
JVP — forward mode

seed $\mathbf{v} \in \mathbb{R}^n$ at the input, result in $\mathbb{R}^m$
directional derivative / Jacobian columns
full Jacobian: about n sweeps

$\mathbf{J}_{\mathbf{f}}^T \mathbf{c}$
VJP — reverse mode

seed $\mathbf{c} \in \mathbb{R}^m$ at the output, result in $\mathbb{R}^n$
pullback / Jacobian rows
full Jacobian: about m sweeps



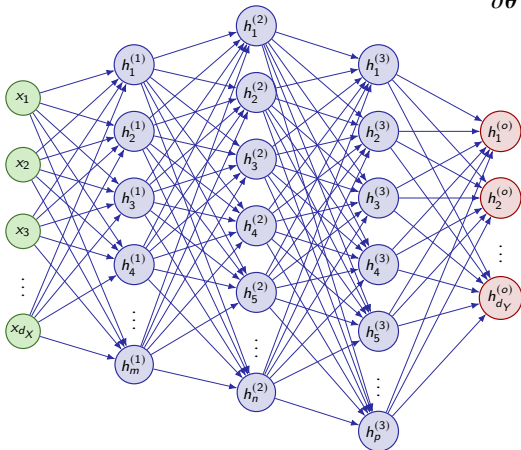
Training neural networks

Backpropagation of the loss: reverse-mode AD

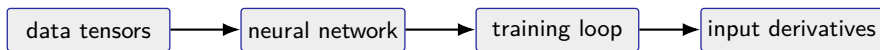
$$L_{\mathcal{D}_{XY}}(\theta) = \frac{1}{N} \sum_{k=1}^N \ell(\mathbf{f}_{\theta}(\mathbf{x}_k), \mathbf{y}_k), \quad \nabla_{\theta} L_{\mathcal{D}_{XY}}(\theta) = \frac{1}{N} \sum_{k=1}^N \nabla_{\theta} \ell(\mathbf{f}_{\theta}(\mathbf{x}_k), \mathbf{y}_k).$$

For a single sample, let $\tilde{\mathbf{y}}_k = \mathbf{f}_{\theta}(\mathbf{x}_k)$. By the chain rule,

$$\nabla_{\theta} \ell(\tilde{\mathbf{y}}_k, \mathbf{y}_k) = [\mathbb{J}_{\theta} \mathbf{f}_{\theta}(\mathbf{x}_k)]^{\top} \nabla_{\tilde{\mathbf{y}}} \ell(\tilde{\mathbf{y}}_k, \mathbf{y}_k), \quad \mathbb{J}_{\theta} \mathbf{f}_{\theta}(\mathbf{x}_k) := \frac{\partial \mathbf{f}_{\theta}(\mathbf{x}_k)}{\partial \theta} \in \mathbb{R}^{d_Y \times p}.$$



PyTorch setup and reproducibility



```
import torch
from torch import nn
from torch.func import jacrev, jacfwd, vmap

torch.set_default_dtype(torch.float64)
torch.manual_seed(7)

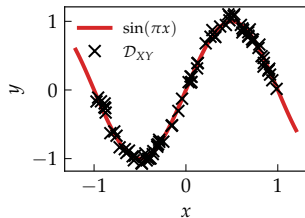
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
```

- ▶ The random seed makes the example repeatable.
- ▶ Double precision is useful when derivatives enter a differential-equation residual.
- ▶ `torch.func` supplies composable derivative transforms.

A synthetic regression dataset

We learn a noisy sample of the smooth function

$$y(x) = \sin(\pi x) + \epsilon, \quad x \in [-1, 1], \quad \epsilon \sim \mathcal{N}(0, 0.05^2).$$



```
N = 80
x_train = 2.0 * torch.rand(N, 1, device=device) - 1.0

def target(x):
    return torch.sin(torch.pi * x)

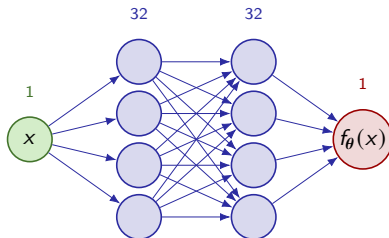
y_clean = target(x_train)
y_train = y_clean + 0.05 * torch.randn_like(y_clean)

x_test = torch.linspace(-1.0, 1.0, 400, device=device).unsqueeze(1)
y_test = target(x_test)
```

The test grid is noise-free and contains points not used by the optimizer.

A neural network in PyTorch

```
model = nn.Sequential(  
    nn.Linear(1, 32),  
    nn.Tanh(),  
    nn.Linear(32, 32),  
    nn.Tanh(),  
    nn.Linear(32, 1),  
) .to(device)  
  
n_parameters = sum(  
    p.numel() for p in model.parameters()  
)  
print(n_parameters)                # 1153
```



Loss function and optimizer

The data loss measures the mean squared prediction error,

$$L_{\mathcal{D}_{XY}}(\theta) = \frac{1}{N} \sum_{i=1}^N |f_{\theta}(x_i) - y_i|^2.$$

```
loss_fn = nn.MSELoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=2.0e-2 # SGD, learning rate = 0.02
)

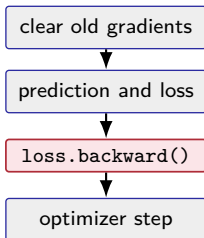
prediction = model(x_train)
loss = loss_fn(prediction, y_train)
```

- ▶ `model.parameters()` exposes the tensors optimized by SGD.
- ▶ the scalar `loss` is the output seed for reverse-mode differentiation.
- ▶ the loss graph connects every parameter to one scalar objective, which is precisely the regime where a VJP is efficient.

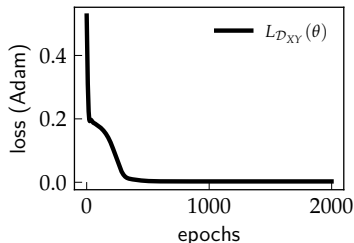
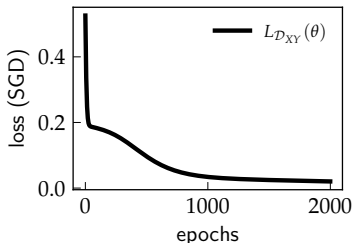
The training loop

```
model.train()
history = []

for step in range(4_000):
    optimizer.zero_grad(set_to_none=True)
    prediction = model(x_train)      # forward pass
    loss = loss_fn(prediction, y_train)
    loss.backward()                  # reverse-mode AD
    optimizer.step()                  # parameter update
    history.append(loss.item())
```



The call to `backward()` accumulates $\nabla_{\theta} L_{\mathcal{D}_{XY}}$ in the `.grad` field of each parameter. We need to clear those fields before the next iteration to prevent unwanted accumulation.



Inspect the trained neural network

```
model.eval()

with torch.no_grad():
    y_test_pred = model(x_test)
    rmse = torch.sqrt(
        loss_fn(y_test_pred, y_test)
    )

assert y_test_pred.shape == y_test.shape
assert torch.isfinite(rmse)
print(f"test RMSE: {rmse.item():.3e}")
```

Evaluation mode

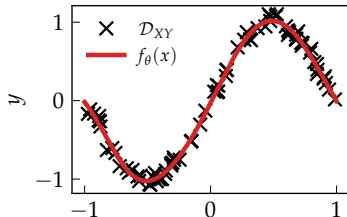
`model.eval()` fixes the behavior of layers such as dropout and batch normalization. It does not disable differentiation by itself.

No derivative graph

`torch.no_grad()` avoids recording operations during this prediction-only check.

Before differentiating the network

Leave the `no_grad` context. Input derivatives require PyTorch to record the operations from x to $f_{\theta}(x)$.



Gradients of a neural network

```
x_phys = torch.linspace(-1.0, 1.0, 200,  
                        device=device).unsqueeze(1)  
x_phys.requires_grad_(True)  
  
u = model(x_phys)  
u_x = torch.autograd.grad(  
    u.sum(), x_phys,  
    create_graph=True  
)[0]
```

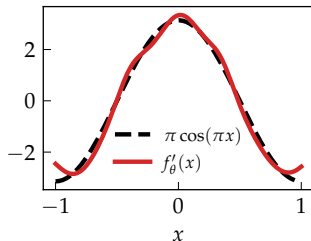
```
def u_single(x):  
    return model(x).squeeze()  
  
u_x_func = vmap(jacrev(u_single))(x_phys.detach())  
  
torch.testing.assert_close(  
    u_x.detach(), u_x_func  
)
```

autograd.grad: the scalar `u.sum()` supplies a cotangent of ones.

jacrev and **vmap**: `jacrev` differentiates one scalar network evaluation, and `vmap` applies that derivative function to all coordinate points.

First-order gradient

$$\nabla_x f_\theta(x) \stackrel{?}{=} \pi \cos(\pi x)$$

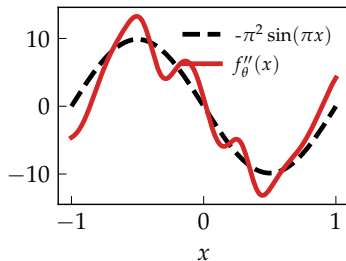


Higher-order gradients of a neural network

The derivative is itself a differentiable program and differentiating it again gives the Hessian.

Second-order gradient

$$\nabla_{xx} f_{\theta}(x) \stackrel{?}{=} -\pi^2 \sin(\pi x)$$



```
grad_u = jacrev(u_single)
hess_u = jacfwd(grad_u)                                # forward over reverse

X = x_phys.detach()
u_value = vmap(u_single)(X)                             # (N,)
H = vmap(hess_u)(X)                                     # (N, 1, 1)
u_xx = H[:, 0, 0]
```

References and further reading

- ▶ A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic Differentiation in Machine Learning: a Survey,” *Journal of Machine Learning Research*, 18:1–43, 2018.
- ▶ L. B. Rall, “Perspectives on Automatic Differentiation: Past, Present, and Future?” in *Automatic Differentiation: Applications, Theory, and Implementations*, Springer, 2006.
- ▶ F. Gatti. (2024). Artificial Neural Networks: Learning the Optimum Statistical Model from Data. In *Machine Learning in Geomechanics 1* (eds I. Stefanou and F. Darve). url: <https://doi.org/10.1002/9781394325665.ch5>.
- ▶ F. Masi (2024). Introduction to Regression Methods. In *Machine Learning in Geomechanics 1* (eds I. Stefanou and F. Darve). url: <https://doi.org/10.1002/9781394325634.ch2>.
- ▶ F. Gatti. (2024). Artificial Neural Networks: Basic architectures and training strategies. In *Machine Learning in Geomechanics 2* (eds I. Stefanou and F. Darve). url: <https://doi.org/10.1002/9781394325665.ch5>.
- ▶ I. Stefanou, F. Darve (2023). ALERT Doctoral School: Machine learning for Geomechanics. url: <https://github.com/alert-geomaterials/2023-doctoral-school..>
- ▶ U. Braga-Neto, *ECEN 744: Scientific Machine Learning, Lecture 3: Automatic Differentiation*, Spring 2026. url: sciml.tamids.tamu.edu/ecen-744-scientific-machine-learning.
- ▶ U. Braga-Neto, *ECEN 744: Scientific Machine Learning, Lecture 4: Physics-Informed Neural Networks*, Spring 2026. url: sciml.tamids.tamu.edu/ecen-744-scientific-machine-learning.